IT4010 Research Project – 2026 Jan

Topic Assessment Form

V2.1

Project ID: R26-ISE-005

1. Topic (12 words max)

An AI-Driven Requirement-Centric System for Intelligent Software Project Requirement Management

2. Research group the project belongs to

SST - Software Systems & Technologies

3. Specialization of the project belongs to

Information System Engineering

4. If a continuation of a previous project:

Project ID	
Year	

5. Brief description of the research problem including references (300 words max) – references not included in word count.

1) Nature and Gravity of the Problem

Requirement Engineering is essential to the success of software projects, but many projects suffer from poor communication between development teams and stakeholders, as well as a lack of domain expertise among technical personnel. Requirements from stakeholders are often expressed in vague or informal language, which can make it challenging to translate them into technical specifications. Additionally, a dependence on static documents like emails or high-level specifications results from low stakeholder engagement. When Domain knowledge is limited, Business Analysts struggle in translating business needs into structured artifacts like Epics, Features, User Stories and Acceptance Criteria.

These difficulties lead to unclear or insufficient requirements, missing functional and non-functional expectations, erroneous presumptions, and poorly organized artifacts. Furthermore, requirement engineers find it difficult to precisely translate business needs into clearly defined Epics, Features, User Stories, and Acceptance Criteria when they lack domain knowledge, especially in specialized domains.

Poorly defined and unprioritized requirements result in confusion, duplicated functionality, and misalignment with stakeholder needs throughout the project lifecycle. When user requirements are expressed inconsistently and without clear priority levels, development teams struggle to identify which features should be implemented first. As a result, critical functionalities may be delayed while less important features receive attention, leading to inefficient resource utilization, frequent rework, increased defects, and uncontrolled scope growth.

Another major challenge is uncontrolled requirement changes and scope creep. Clients may request new features or modify existing requirements without a structured change management process. Without proper tracking, approval workflows, and impact analysis, teams cannot clearly identify how changes affect project scope, cost, timeline, and resources. This often leads to project delays, budget overruns, reduced team morale, and decreased client satisfaction.

The gravity of this problem is high, as poor requirements management is widely recognized as a leading cause of software project failure, particularly in large-scale and public-sector projects. Addressing issues such as unclear communication, weak documentation, and unmanaged requirement changes is essential to improve software project success, transparency, and efficiency.

The problem mainly affects software development teams and organizations involved in requirement-driven projects. Key groups impacted include Business Analysts, Software Engineers, Project Managers, and clients, who rely on accurate requirement understanding and project alignment. Clients often lack technical knowledge and provide incomplete or ambiguous requirements, while development teams may have limited domain knowledge. This creates communication gaps, unclear specifications, frequent changes, and poor alignment between requirements and project plans. As a result, projects suffer from delays, rework, cost overruns, and reduced client satisfaction. The gravity of the problem is high, as poor requirements management is a major cause of software project failure, particularly in large-scale and public-sector projects.

References

[1] K. E. Wiegers and J. Beatty, Software Requirements, 3rd ed. Redmond, WA, USA: Microsoft Press, 2013.

[2] A. Ferrari, G. O. Spagnolo, and F. Dell'Orletta, "The impact of requirements documentation quality on software development," in Proc. IEEE Int. Requirements Engineering Conf. (RE), 2016, pp. 1–10.

[3] K. K. Agrawal and S. K. Khatri, "Impact of requirement volatility on software project success," Int. J. Software Engineering & Applications (IJSEA), vol. 8, no. 2, pp. 1–14, 2017.

[4] V. Lenarduzzi, A. Martini, D. Taibi, and D. Fucci, "Technical debt and agile software development," in Proc. Agile Conf., 2015, pp. 1–10.

6. Brief description of Existing Research and Systems (300 words max)

Summary of Existing Products / Systems

There have been existing systems for the management of requirements as well as project management in place that have been widely adopted in both industry as well as academic institutions to aid in software development tasks. These tools include Jira, Azure DevOps, IBM DOORS, Confluence, Trello, as well as Asana. Research shows that the tools are useful in tracking tasks as well as monitoring progress after the implementation of requirements but presume the requirements are structured before inputting them into the system [1], [2].

IBM DOORS is renowned as a reliable requirements management solution that provides traceability, versioning, and change history, primarily in the realm of complex and safety-critical applications [3]. However, despite the benefits it provides, the need for manual intensive work, in-depth domain knowledge, and the use of proper requirement specifications in a research study indicates that the application of DOORS is less desirable in the early stages of requirement gathering and in agile projects [4]. Jira and Azure DevOps, in a similar manner, use user stories, backlogging, and change history, with a strong reliance on manual work, analyst expertise, and human prioritization of the conversion of stakeholder needs into Epics, Features, and Acceptance Criteria, respectively [5].

Studies have revealed that the majority of commercial software does little for requirement clarification, ambiguity identification, or domain knowledge support during the requirement capture process [6]. Requirements are mostly expressed as natural language text with the absence of automated validation, causing these requirements to become ambiguous, incomplete, or incorrectly understood. This is supported by Ferrari et al., who show that low-quality requirement writing has a substantial impact on rework rates and defective units; however, little intelligence is offered by current software for requirement quality assessment during entry [7].

There have been a few research endeavors trying to apply AI-supported, as well as NLP-enabled, techniques in the realm of requirements engineering, related to ambiguity detection, requirement assignment, traceability analysis, and effort prediction [8], [9]. Yet, the results obtained in these research versions are encouraging, but they are always developed in a disconnected manner. Additionally, the current state-of-the-art tools lack workflow integrity in terms of end-to-end analysis involving understanding, prioritization, planning, as well as impact analysis in the realm of requirements [10].

Overall, the literature shows one major limitation: that requirements, documentation, prioritization, project planning, and change management are hosted in disjoined activities. Neither research nor industry solutions support an integrated, requirement-centric, and intelligent system to actively support stakeholders from the very first requirement elicitation throughout project execution to the controlled execution of changes.

[1] K. Wiegers and J. Beatty, *Software Requirements*, 3rd ed. Redmond, WA, USA: Microsoft Press, 2013.
[2] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.
[3] A. van Lamsweerde, *Requirements Engineering: From System Goals to UML Models to Software Specifications*. Chichester, U.K.: Wiley, 2009.
[4] B. Boehm and R. Turner, "Balancing agility and discipline: A guide for the perplexed," Addison-Wesley, 2004.
[5] M. Paetsch, A. Eberlein, and F. Maurer, "Requirements engineering and agile software development," *Proc. IEEE Int. Workshop on Enabling Technologies*, 2003, pp. 308–313.
[6] D. Zowghi and C. Coulin, "Requirements elicitation: A survey of techniques, approaches, and tools," in *Engineering and Managing Software Requirements*, Springer, 2005, pp. 19–46.
[7] A. Ferrari et al., "The impact of requirements documentation quality on software development," in *Proc. IEEE Int. Conf. on Requirements Engineering (RE)*, 2016, pp. 1–10.
[8] N. Dalpiaz, F. Aydemir, and P. Giorgini, "Detecting ambiguities in natural language requirements," *Proc. IEEE RE*, 2012.
[9] V. Lenarduzzi et al., "Technical debt in agile software development: A systematic mapping study," *Agile Conf.*, 2015.
[10] K. K. Agrawal and S. K. Khatri, "Impact of requirement volatility on software project success," *Int. J. Software Engineering & Applications*, vol. 8, no. 3, pp. 1–14, 2017.

Gaps and Limitations Identified

Despite the availability of tools and research, several gaps remain. Existing products do not automatically transform requirements into project plans or intelligently assess the impact of requirement changes. Documentation features are underutilized due to manual formatting requirements and the lack of lightweight, user-friendly systems. Change management is mostly manual, with limited support for automated impact analysis, proactive alerts for scope creep, or guided approval workflows. Research solutions are often experimental, complex, or impractical for real project workflows. Additionally, most systems treat requirements, documentation, and project management as separate activities, leading to fragmentation and poor integration. Overall, there is limited support for real-time miscommunication detection, requirement-centric progress visibility, and proactive risk management.

User Story Prioritization Issue

Existing tools such as Jira, Confluence, Trello, and Azure DevOps support user story management and backlog tracking, but they often rely heavily on manual prioritization and subjective decision-making. In many projects, user stories are created inconsistently by different stakeholders, and prioritization is performed informally without a structured framework, especially during early project stages. Existing research emphasizes the importance of user stories in agile development and proposes prioritization techniques such as MoSCoW and Kano models; however, practical integration of these techniques into automated and easy-to-use systems remains limited. Identified research gaps include the lack of systematic prioritization logic that can aggregate user stories from multiple contributors, limited automation in scheduling implementation order based on priority and dependency, insufficient visual support for priority-based decision-making, and inadequate mechanisms to prevent scope creep in small to medium-scale development teams.

Requirement Changes and Scope Creep.

Existing tools like Jira, Trello, Asana, and Microsoft DevOps allow for tracking of changes but often lack built-in change management workflows. They rely on manual processes for impact analysis and approval routing, with no native mechanism to detect scope creep or prioritize change requests intelligently. Existing research focuses on process models, case studies, and post-mortem analysis of requirement volatility, but practical, integrated solutions that automate impact assessment and decision support are limited. Research gaps include real-time impact scoring, predictive alerting for scope creep, and guided approval workflows. Key limitations identified include the absence of automatic change impact classification, poor integration with approval governance, minimal decision support for change reviewers, and lack of proactive alerts to prevent uncontrolled scope expansion.

Existing Research / Tool	Focus / Features	Limitations / Gaps	Proposed System Contribution
IBM DOORS [3]	Requirement traceability, versioning, formal specification management	Requires extensive domain knowledge, not suitable for informal or early-stage requirements; limited automation in requirement clarification	Automatically interprets unstructured requirements using NLP, resolves ambiguities, and augments domain knowledge
Jira / Azure DevOps [5]	Backlog management, user stories, task tracking	Manual prioritization, relies on analyst expertise; lacks intelligent guidance for converting stakeholder input into Epics, Features, Stories	Generates structured Agile artifacts from informal stakeholder inputs, includes automated capacity estimation and Jira integration
Confluence / Trello / Asana [1], [2]	Documentation, task boards, progress tracking	Documentation underutilized; informal change tracking; poor integration with project planning	Centralized documentation, intelligent linking of requirements to project tasks, automated change impact assessment
AI-assisted Requirement Research (e.g., Dalpiaz et al., 2012 [8]; Ferrari et al., 2016 [7])	Ambiguity detection, requirement classification, impact estimation	Focus on isolated tasks; not integrated with project management platforms; limited real-world applicability	End-to-end integration: requirement processing, prioritization, project planning, and change management in one platform
Proposed System (This Research)	Requirement-driven project management; NLP-based requirement clarification; automated user story generation; MoSCoW prioritization; requirement-level time/resource planning; scope creep detection	N/A	Fills gaps in current tools and research by combining requirement analysis, prioritization, planning, and change management; supports early-stage unstructured inputs and real-time project monitoring

7. Brief description of the solution's nature, including a conceptual diagram (maximum 500 words).

Addressing the Identified Gaps

The proposed system addresses the gaps in existing tools and research by providing an integrated, requirement-driven, and intelligent platform for requirements engineering and project management. Existing products treat requirements, documentation, project planning, and change management as separate activities, while research solutions often focus on isolated problems such as ambiguity detection or effort estimation. This system unifies all requirement-related activities, automates requirement clarification, links requirements to project tasks, centralizes documentation, and manages requirement changes with impact awareness. It reduces manual effort, improves traceability, minimizes miscommunication, and supports proactive decision-making, making it practical for real-world software projects.

System Perspective: Input , Process , Output

1.Requirement Processing and Jira Integration Component
Input: Unstructured client requirements, project context.
Process: The AI interprets requirements using NLP, resolves ambiguities, augments domain knowledge, and generates Epics, Features, User Stories, Acceptance Criteria, and capacity estimates, then integrates them with Jira.
Output: Structured Agile artifacts, clear acceptance criteria, capacity insights, and automatically update Jira.

2.User Story Prioritization Component
Input: User stories created by different team members, representing system requirements from various stakeholder perspectives.
Process: Collection of user stories into a centralized backlog, prioritization using the MoSCoW method (Must Have, Should Have, Could Have, Won't Have), analysis of dependencies, and scheduling of implementation order based on priority and project constraints.
Output:A prioritized and ordered list of user stories displayed through a clear user interface, enabling the development team to focus on high-priority requirements first and supporting effective scope control

3.Requirement-Driven Project Management for Time and Resource Optimization
Input:
Validated software requirements, defined project time constraints, available resource profiles (human and technical), requirement dependencies, and approved requirement change requests.
Process:
This component performs requirement-level analysis to assess complexity, priority, and interdependencies that influence time estimation and resource demand. Based on this analysis, requirements are systematically transformed into executable project elements with associated effort estimates. The system then applies requirement-driven planning logic to generate project schedules, assign resources, and distribute workloads in accordance with availability and capacity constraints.
During project execution, the component continuously monitors requirement-linked progress to identify schedule deviations and resource utilization inefficiencies. When requirements are modified, the component automatically recalculates timeline projections and resource allocations to reflect the updated effort and dependency structure.
Output:
The output includes requirement-aligned project timelines, optimized resource allocation plans, and real-time visibility into progress, delays, and workload distribution. This enables proactive management of delivery schedules and resource usage, improving planning accuracy and execution predictability throughout the software development lifecycle.

4. Requirement Change & Scope Creep Component

Input: Change requests including description, priority, reason, requester, affected modules, and project constraints.
Process: Automated impact scoring (Low/Medium/High), approval workflow routing (Approve/Reject/Needs Clarification), audit logging, and monitoring for abnormal patterns to detect scope creep.
Output: Approval/rejection decisions, impact reports, change history logs, and real-time alerts for potential scope creep.

Integration Summary:
The system integrates documentation, change management, project planning, and intelligent monitoring into a single platform. Requirements are captured, validated, and shared clearly; changes are evaluated and controlled; tasks are aligned with requirements; and real-time monitoring provides proactive risk detection and decision support. This ensures transparency, accountability, and efficient execution across the software project lifecycle

8. Brief overview of the availability of necessary specialized domain expertise, knowledge, and data. (500 words max)

Gaining Essential Specialized Domain Expertise

Members of the project team gain essential domain expertise through a combination of structured learning and practical exposure. This includes reviewing academic literature, industry articles, and best-practice guidelines related to requirements engineering, project management, and software development processes. Team members study real-world project documentation such as requirement specifications, change request forms, and project plans to understand how requirements are handled in professional environments. Additionally, discussions with lecturers, supervisors, and industry practitioners help clarify domain-specific concepts, workflows, and common challenges. By analyzing real project scenarios and case studies, team members develop a practical understanding of how communication gaps, scope creep, and poor documentation affect software projects. For the User Story Prioritization component, the team studies agile requirements engineering practices, including user story formulation and prioritization techniques such as the MoSCoW method, and reviews real-world agile backlogs to understand best practices in defining, prioritizing, and scheduling user stories. For Requirement Changes and Scope Creep, the team examines case studies and datasets to understand how changes are tracked, evaluated, and managed in real projects.

Gaining Technical Knowledge and Skills

The technical knowledge required for the project is acquired through hands on learning and tool-based practice. Team members strengthen their skills in areas such as system design, database management, and software development using appropriate programming languages and frameworks. Online learning platforms, official documentation, tutorials, and sample projects are used to learn technologies related to requirement management systems, project tracking tools, and basic intelligent features. Practical implementation tasks such as building prototypes, designing system architectures, and developing user interfaces allow members to apply theoretical knowledge in a real context. Regular testing, debugging, and iterative improvement help improve problem-solving skills and technical confidence. For the User Story Prioritization component, technical skills related to web application development, database management, and implementation of rule-based prioritization logic are acquired through academic coursework, online tutorials, and practical experimentation. Additional knowledge is gained in designing user interfaces to visualize prioritized user stories and implementation order effectively. For Requirement Changes, knowledge in workflow automation, database handling, alerting systems, and basic AI or rule-based classifiers is learned to assess change impact and detect potential scope creep. Practical implementation, testing, debugging, and iterative improvement reinforce technical understanding and problem-solving skills

Data Collection for the Project.

The data required for this project is collected using secondary and simulated sources. The data is defined and organized according to each system component to ensure clarity and proper evaluation. No sensitive, personal, or confidential real-world data is used in any component.

1. Requirement Processing and Jira Integration Component

Required Data

To address issues related to vague stakeholder communication, lack of domain knowledge, and difficulty in translating business needs into structured Agile artifacts, the following data is required for this component:

- **Unstructured Client Requirements**
  Natural-language requirement descriptions expressed in informal formats such as paragraphs, bullet points, emails, meeting notes, and high-level specifications from BAs or PMs. These inputs represent how stakeholders typically communicate requirements without technical structure.

- **Domain Context Information**
  Background knowledge related to the application domain (e.g., business processes, common terminologies, constraints, and typical functional patterns) to support domain understanding and reduce misinterpretation during requirement analysis.
- **Sample Agile Artifacts**
  Examples of Epics, Features, User Stories, and Acceptance Criteria used as reference structures for generating consistent and well-formed outputs.
- **Capacity and Effort Estimation References**
  Basic estimation data such as relative story sizes, complexity indicators, or sample sprint capacities to support capacity estimation during artifact generation.
- **Jira Project Metadata**
  Jira project configurations including project keys, issue types (Epic, Story, Task), workflows, and required fields to enable correct integration and automated artifact creation.

Ethical Clearance Considerations

This project does not require formal ethical clearance because it does not involve human subjects, personal data collection, medical data, or sensitive information. The system does not perform experiments on users or collect identifiable personal data. All data used for development and testing is either simulated, publicly available, or anonymized. Ethical principles such as transparency, responsible system design, and respect for professional standards are still followed throughout the project. If user feedback is collected informally for evaluation purposes, it will be voluntary and non-identifiable, ensuring ethical integrity.

9. Objectives and Novelty

Main Objective

This research project aims to design and develop an intelligent, requirement-driven system that improves requirement understanding, documentation, project planning, and change management in software projects. The study focuses on reducing miscommunication, rework, and scope creep by integrating automated requirement analysis with project management support. Ultimately, the research seeks to enhance software project efficiency, transparency, and decision-making.

Member Name with Registration No	Sub Objective	Tasks	Novelty
IT22214652	To implement an AI-driven requirement processing module that automatically interprets unstructured client requirements, detects ambiguities and inconsistencies, augments domain knowledge, and generates structured Agile artifacts (Epics, Features, User Stories, and Acceptance Criteria), ensuring improved clarity, traceability, and actionable outputs for software project planning.	1. Collect and preprocess unstructured client requirements. 2. Interpret requirements using NLP to detect ambiguities and extract functional elements. 3. Augment understanding with domain-specific knowledge. 4. Generate Epics, Features, User Stories, and Acceptance Criteria. 5. Estimate development capacity for sprint planning. 6. Integrate generated artifacts into Jira for Agile workflow management.	Automatically transforms unstructured client requirements into structured Agile artifacts like Epics, Features, User Stories and Acceptance Criteria with capacity estimates.
IT22139580	To implement a rule-based user story prioritization and scheduling module using the MoSCoW method, which organizes user stories into a clear implementation order to support effective	1. Collect user stories created by different team members. 2. Apply the MoSCoW prioritization technique to classify user stories. 3. Analyze basic dependencies between user stories. 4. Schedule the implementation order based on priority levels.	Automated prioritization and scheduling of user stories using a structured framework, providing clear visual guidance for development order and supporting scope control.

	development planning and scope control.	5. Present prioritized user stories through a clear and user-friendly interface	
IT22134844	To implement a requirement-driven project management module that utilizes prioritized requirements to support effective time planning and efficient resource allocation.	To implement this component and support the overall research objectives, the following tasks are performed: 1. Requirement-to-Execution Mapping Design mechanisms to automatically link approved requirements to project tasks and milestones. Maintain traceability between requirements, timelines, and assigned resources. 2. Time Estimation Based on Requirement Complexity Analyze requirement attributes such as complexity, priority, and dependencies. Generate realistic effort and duration estimates for each requirement-driven task. 3. Resource Allocation and Workload Distribution Assign resources (developers, testers, analysts) based on requirement effort and availability. Balance workloads to prevent over-allocation and idle resources. 4. Real-Time Progress Monitoring Track implementation progress at the requirement level throughout the project lifecycle. Identify delays, blocked tasks, and execution bottlenecks early. 5. Change Impact Analysis (Time & Resources) Evaluate how requirement changes affect delivery timelines and resource workloads. Provide updated schedules and workload forecasts after changes. 6. Decision-Support Intelligence Detect scheduling conflicts and resource overload automatically.	Requirement-Centric Scheduling Time and resource planning is driven directly by individual requirements, not abstract tasks. Dynamic Time & Resource Recalculation Any requirement change automatically triggers updated timeline and workload impact analysis. Real-Time Requirement-Level Visibility Progress, delays, and bottlenecks are tracked at the requirement level, which is rarely addressed in existing systems. Integrated Intelligence, Not Standalone Estimation Instead of isolated effort estimation models, this component embeds intelligent time and resource control into the execution workflow.

		Predict potential delivery risks caused by frequent or high-impact requirement changes.	Practical, End-to-End Applicability Unlike many research prototypes that focus on isolated algorithms, this component is designed to be usable in real software projects.
IT22115416	To implement a rule-based Requirement Change Management System that manages, evaluates, and approves requirement changes in order to control scope creep throughout the project lifecycle.	**1.** Design and develop a change request submission and tracking interface. **2.** - Implement an automated impact scoring system based on priority, complexity, and affected modules. **3.** - Develop an approval workflow with role-based routing and decision logging. **4.** - Create real-time alerts for abnormal change frequency and scope creep detection.	• Automatic impact level indicator (Low/Medium/High) based on priority, complexity, and affected modules. • Smart decision guidance with suggestions: Approve, Reject, or Needs Clarification. • Real-time scope creep detection via change pattern analysis and alerts. • Audit trail and accountability tracking with timestamped decisions and user logs.

10. This part is to be filled by the Supervisor and the Co-supervisor of the Project.

a) This research topic possesses a comprehensive scope suitable for a final-year project.

Yes	✓	No	

b) The proposed topic exhibits novelty.

Yes	✓	No	

c) The student group can successfully execute the proposed project.

Yes	✓	No	

d) The proposed sub-objectives reflect the students' areas of specialization.

Yes	✓	No	

e) Any other comments:

Need to do some changes in the document to update components.
Need a good lit. review.

	Title	First Name	Last Name	Signature
Supervisor	Mrs.	Budathima	Attanayaka	
Co-Supervisor	Ms	Chathurangika	Kahandawaarachchi	
External Supervisor				
Summary of external supervisor's (if any) experience and expertise				